

```
"name": "ai-healthcare-kiosk",
"version": "1.0.0",
"description": "AI-driven preventive healthcare kiosk system",
"main": "server.js",
"scripts": {
  "start": "node server.js",
  "dev": "nodemon server.js",
  "client": "cd client && npm start",
  "build": "cd client && npm run build",
  "install-all": "npm install && cd client && npm install"
},
"dependencies": {
  "express": "^4.18.2",
  "cors": "^2.8.5",
  "helmet": "^7.1.0",
  "dotenv": "^16.3.1",
  "mongoose": "^8.0.3",
  "bcryptjs": "^2.4.3",
  "jsonwebtoken": "^9.0.2",
  "multer": "^1.4.5-lts.1",
  "axios": "^1.6.2",
  "socket.io": "^4.7.4",
  "express-rate-limit": "^7.1.5",
  "compression": "^1.7.4",
  "morgan": "^1.10.0"
},
"devDependencies": {
  "nodemon": "^3.0.2"
```

```
},  
  "keywords": [  
    "healthcare",  
    "ai",  
    "kiosk",  
    "preventive",  
    "health-assessment"  
  ],  
  "author": "AI Healthcare Kiosk Team",  
  "license": "MIT"  
}
```

Server Configuration

PORT=5000

NODE_ENV=development

Database

MONGODB_URI=mongodb://localhost:27017/healthcare-kiosk

JWT Secret

JWT_SECRET=your-super-secret-jwt-key-here

AI Service Configuration

OPENAI_API_KEY=your-openai-api-key-here

HEALTH_AI_SERVICE_URL=https://api.health-ai-service.com

Kiosk Configuration

KIOSK_ID=kiosk-001

KIOSK_LOCATION="Main Hospital Lobby"

MAX_SESSION_DURATION=1800000

Security

RATE_LIMIT_WINDOW=15

RATE_LIMIT_MAX_REQUESTS=100

File Upload

MAX_FILE_SIZE=10485760

UPLOAD_PATH=./uploads

const express = require('express');

const cors = require('cors');

const helmet = require('helmet');

const compression = require('compression');

const morgan = require('morgan');

const rateLimit = require('express-rate-limit');

const path = require('path');

const http = require('http');

const socketIo = require('socket.io');

require('dotenv').config();

const **connectDB** = require('./config/database');

const authRoutes = require('./routes/auth');

const healthRoutes = require('./routes/health');

const kioskRoutes = require('./routes/kiosk');

const aiRoutes = require('./routes/ai');

const { **errorHandler** } = require('./middleware/errorHandler');

const app = express();

```
const server = http.createServer(app);

const io = socketIo(server, {
  cors: {
    origin: process.env.CLIENT_URL || "http://localhost:3000",
    methods: ["GET", "POST"]
  }
});
```

```
// Connect to MongoDB
```

```
connectDB();
```

```
// Security middleware
```

```
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      styleSrc: ["'self'", "unsafe-inline"],
      scriptSrc: ["'self'"],
      imgSrc: ["'self'", "data:", "https:"],
    },
  },
}));
```

```
// Rate limiting
```

```
const limiter = rateLimit({
  windowMs: (process.env.RATE_LIMIT_WINDOW || 15) * 60 * 1000, // 15 minutes
  max: process.env.RATE_LIMIT_MAX_REQUESTS || 100,
  message: 'Too many requests from this IP, please try again later.'
```

```
});  
  
app.use('/api/', limiter);  
  
// Middleware  
  
app.use(compression());  
app.use(morgan('combined'));  
app.use(cors({  
  origin: process.env.CLIENT_URL || "http://localhost:3000",  
  credentials: true  
}));  
  
app.use(express.json({ limit: '10mb' }));  
app.use(express.urlencoded({ extended: true, limit: '10mb' }));  
  
// Static files  
  
app.use('/uploads', express.static(path.join(__dirname, 'uploads')));  
  
// Routes  
  
app.use('/api/auth', authRoutes);  
app.use('/api/health', healthRoutes);  
app.use('/api/kiosk', kioskRoutes);  
app.use('/api/ai', aiRoutes);  
  
// Health check endpoint  
  
app.get('/api/health-check', (req, res) => {  
  res.json({  
    status: 'OK',  
    timestamp: new Date().toISOString(),  
    kioskId: process.env.KIOSK_ID,
```

```
    version: '1.0.0'

  });

});

// Serve static files from React app in production
if (process.env.NODE_ENV === 'production') {

  app.use(express.static(path.join(__dirname, 'client/build')));

  app.get('*', (req, res) => {

    res.sendFile(path.join(__dirname, 'client/build', 'index.html'));

  });

}

// Socket.IO for real-time communication

io.on('connection', (socket) => {

  console.log('Kiosk client connected:', socket.id);

  socket.on('health-assessment-start', (data) => {

    console.log('Health assessment started:', data);

    socket.broadcast.emit('assessment-in-progress', data);

  });

  socket.on('health-assessment-complete', (data) => {

    console.log('Health assessment completed:', data);

    socket.broadcast.emit('assessment-completed', data);

  });

  socket.on('disconnect', () => {
```

```
    console.log('Kiosk client disconnected:', socket.id);
  });
});

// Error handling middleware
app.use(errorHandler);

// 404 handler
app.use('*', (req, res) => {
  res.status(404).json({
    success: false,
    message: 'Route not found'
  });
});

const PORT = process.env.PORT || 5000;

server.listen(PORT, () => {
  console.log(` 🚀 AI Healthcare Kiosk Server running on port ${PORT} `);
  console.log(` 🧩 Kiosk ID: ${process.env.KIOSK_ID || 'kiosk-001'} `);
  console.log(` 🌍 Environment: ${process.env.NODE_ENV || 'development'} `);
});

module.exports = { app, io };

const mongoose = require('mongoose');

const connectDB = async () => {
```

```
try {  
  const conn = await mongoose.connect(process.env.MONGODB_URI ||  
'mongodb://localhost:27017/healthcare-kiosk', {  
    useNewUrlParser: true,  
    useUnifiedTopology: true,  
  });  
  
  console.log(`🟢 MongoDB Connected: ${conn.connection.host}`);  
  
  // Connection event listeners  
  mongoose.connection.on('error', (err) => {  
    console.error('MongoDB connection error:', err);  
  });  
  
  mongoose.connection.on('disconnected', () => {  
    console.log('MongoDB disconnected');  
  });  
  
  // Graceful shutdown  
  process.on('SIGINT', async () => {  
    await mongoose.connection.close();  
    console.log('MongoDB connection closed through app termination');  
    process.exit(0);  
  });  
  
} catch (error) {  
  console.error('Database connection failed:', error.message);  
  process.exit(1);  
}
```

```
}  
};
```

```
module.exports = connectDB;
```

```
const mongoose = require('mongoose');
```

```
const bcrypt = require('bcryptjs');
```

```
const userSchema = new mongoose.Schema({
```

```
  email: {  
    type: String,  
    required: true,  
    unique: true,  
    lowercase: true,  
    trim: true
```

```
  },
```

```
  password: {  
    type: String,  
    required: true,  
    minlength: 6
```

```
  },
```

```
  firstName: {  
    type: String,  
    required: true,  
    trim: true
```

```
  },
```

```
  lastName: {  
    type: String,
```

```
    required: true,

    trim: true
  },
  dateOfBirth: {
    type: Date,
    required: true
  },
  gender: {
    type: String,
    enum: ['male', 'female', 'other', 'prefer-not-to-say'],
    required: true
  },
  phoneNumber: {
    type: String,
    trim: true
  },
  emergencyContact: {
    name: String,
    phone: String,
    relationship: String
  },
  medicalHistory: [{
    condition: String,
    diagnosedDate: Date,
    status: {
      type: String,
      enum: ['active', 'resolved', 'chronic'],
      default: 'active'
    }
  }]
```

```
    }
  }],
  medications: [{
    name: String,
    dosage: String,
    frequency: String,
    startDate: Date
  }],
  allergies: [{
    allergen: String,
    severity: {
      type: String,
      enum: ['mild', 'moderate', 'severe'],
      default: 'mild'
    }
  }],
  role: {
    type: String,
    enum: ['patient', 'admin', 'healthcare_provider'],
    default: 'patient'
  },
  isActive: {
    type: Boolean,
    default: true
  },
  lastLogin: {
    type: Date,
    default: Date.now
  }
}
```

```
},
preferences: {
  language: {
    type: String,
    default: 'en'
  },
  notifications: {
    email: {
      type: Boolean,
      default: true
    },
    sms: {
      type: Boolean,
      default: false
    }
  }
}, {
  timestamps: true
});
```

// Index for better query performance

```
userSchema.index({ email: 1 });
```

```
userSchema.index({ createdAt: -1 });
```

// Hash password before saving

```
userSchema.pre('save', async function(next) {
  if (!this.isModified('password')) return next();
```

```
try {  
  const salt = await bcrypt.genSalt(12);  
  this.password = await bcrypt.hash(this.password, salt);  
  next();  
} catch (error) {  
  next(error);  
}  
});
```

// Compare password method

```
userSchema.methods.comparePassword = async function(candidatePassword) {  
  return bcrypt.compare(candidatePassword, this.password);  
};
```

// Get user's full name

```
userSchema.virtual('fullName').get(function() {  
  return `${this.firstName} ${this.lastName}`;  
});
```

// Remove password from JSON output

```
userSchema.methods.toJSON = function() {  
  const user = this.toObject();  
  delete user.password;  
  return user;  
};
```

```
module.exports = mongoose.model('User', userSchema);
```

```
const mongoose = require('mongoose');

const vitalSignsSchema = new mongoose.Schema({
  bloodPressure: {
    systolic: { type: Number, min: 50, max: 300 },
    diastolic: { type: Number, min: 30, max: 200 }
  },
  heartRate: {
    type: Number,
    min: 30,
    max: 220
  },
  temperature: {
    type: Number,
    min: 95,
    max: 110
  },
  oxygenSaturation: {
    type: Number,
    min: 70,
    max: 100
  },
  weight: {
    type: Number,
    min: 20,
    max: 500
  },
});
```

```
height: {  
  type: Number,  
  min: 100,  
  max: 250  
},  
bmi: {  
  type: Number,  
  min: 10,  
  max: 80  
}  
});
```

```
const symptomsSchema = new mongoose.Schema({  
  primarySymptoms: [{  
    symptom: String,  
    severity: {  
      type: String,  
      enum: ['mild', 'moderate', 'severe'],  
      default: 'mild'  
    },  
    duration: String,  
    frequency: String  
  }],  
  additionalSymptoms: [String],  
  painLevel: {  
    type: Number,  
    min: 0,  
    max: 10
```

```
},  
painLocation: [String],  
onset: {  
  type: String,  
  enum: ['sudden', 'gradual', 'intermittent'],  
  default: 'gradual'  
}  
});
```

```
const lifestyleSchema = new mongoose.Schema({  
  exerciseFrequency: {  
    type: String,  
    enum: ['none', 'light', 'moderate', 'intense'],  
    default: 'none'  
  },  
  dietQuality: {  
    type: String,  
    enum: ['poor', 'fair', 'good', 'excellent'],  
    default: 'fair'  
  },  
  sleepHours: {  
    type: Number,  
    min: 0,  
    max: 24  
  },  
  stressLevel: {  
    type: String,  
    enum: ['low', 'moderate', 'high', 'very-high'],
```

```
    default: 'moderate'
  },
  smokingStatus: {
    type: String,
    enum: ['never', 'former', 'current'],
    default: 'never'
  },
  alcoholConsumption: {
    type: String,
    enum: ['none', 'light', 'moderate', 'heavy'],
    default: 'none'
  }
});
```

```
const aiAnalysisSchema = new mongoose.Schema({
  riskScore: {
    type: Number,
    min: 0,
    max: 100
  },
  riskFactors: [String],
  recommendations: [{
    category: String,
    priority: {
      type: String,
      enum: ['low', 'medium', 'high', 'urgent'],
      default: 'medium'
    }
  }],
});
```

```
    title: String,
    description: String,
    actionItems: [String]
  }},
  predictedConditions: [{
    condition: String,
    probability: Number,
    confidence: String
  }],
  followUpRequired: {
    type: Boolean,
    default: false
  },
  followUpTimeframe: String,
  urgencyLevel: {
    type: String,
    enum: ['low', 'medium', 'high', 'critical'],
    default: 'low'
  }
});
```

```
const healthAssessmentSchema = new mongoose.Schema({
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  kioskId: {
```

```
    type: String,
    required: true
  },
  sessionId: {
    type: String,
    required: true,
    unique: true
  },
  assessmentType: {
    type: String,
    enum: ['general', 'cardiovascular', 'respiratory', 'diabetes', 'mental-health',
'comprehensive'],
    default: 'general'
  },
  vitalSigns: vitalSignsSchema,
  symptoms: symptomsSchema,
  lifestyle: lifestyleSchema,
  questionnaire: {
    responses: [{
      questionId: String,
      question: String,
      answer: mongoose.Schema.Types.Mixed,
      timestamp: {
        type: Date,
        default: Date.now
      }
    }]
  },
```

```
aiAnalysis: aiAnalysisSchema,
status: {
  type: String,
  enum: ['in-progress', 'completed', 'reviewed', 'archived'],
  default: 'in-progress'
},
completedAt: Date,
reviewedBy: {
  type: mongoose.Schema.Types.ObjectId,
  ref: 'User'
},
reviewedAt: Date,
notes: String,
attachments: [{
  filename: String,
  originalName: String,
  mimetype: String,
  size: Number,
  uploadedAt: {
    type: Date,
    default: Date.now
  }
}]
}, {
  timestamps: true
});
```

// Indexes for better query performance

```
healthAssessmentSchema.index({ userId: 1, createdAt: -1 });
healthAssessmentSchema.index({ kioskId: 1, createdAt: -1 });
healthAssessmentSchema.index({ sessionId: 1 });
healthAssessmentSchema.index({ status: 1 });
healthAssessmentSchema.index({ 'aiAnalysis.urgencyLevel': 1 });
```

// Virtual for assessment duration

```
healthAssessmentSchema.virtual('duration').get(function() {
  if (this.completedAt && this.createdAt) {
    return this.completedAt - this.createdAt;
  }
  return null;
});
```

// Method to calculate BMI if not provided

```
healthAssessmentSchema.methods.calculateBMI = function() {
  if (this.vitalSigns.weight && this.vitalSigns.height) {
    const heightInMeters = this.vitalSigns.height / 100;
    return (this.vitalSigns.weight / (heightInMeters * heightInMeters)).toFixed(1);
  }
  return null;
};
```

// Method to get risk level based on risk score

```
healthAssessmentSchema.methods.getRiskLevel = function() {
  const score = this.aiAnalysis?.riskScore || 0;
  if (score >= 80) return 'critical';
  if (score >= 60) return 'high';
```

```
    if (score >= 40) return 'medium';  
    return 'low';  
};
```

```
module.exports = mongoose.model('HealthAssessment', healthAssessmentSchema);
```

```
const mongoose = require('mongoose');
```

```
const kioskSessionSchema = new mongoose.Schema({  
  sessionId: {  
    type: String,  
    required: true,  
    unique: true  
  },  
  kioskId: {  
    type: String,  
    required: true  
  },  
  userId: {  
    type: mongoose.Schema.Types.ObjectId,  
    ref: 'User'  
  },  
  status: {  
    type: String,  
    enum: ['active', 'completed', 'abandoned', 'error'],  
    default: 'active'  
  },  
  startTime: {
```

```
  type: Date,
  default: Date.now
},
endTime: Date,
duration: Number, // in milliseconds
stepsCompleted: [{
  step: String,
  completedAt: Date,
  data: mongoose.Schema.Types.Mixed
}],
currentStep: {
  type: String,
  default: 'welcome'
},
deviceInfo: {
  userAgent: String,
  screenResolution: String,
  touchEnabled: Boolean,
  browserVersion: String
},
location: {
  ip: String,
  country: String,
  city: String,
  coordinates: {
    lat: Number,
    lng: Number
  }
}
```

```
},
errors: [{
  step: String,
  error: String,
  timestamp: {
    type: Date,
    default: Date.now
  }
}],
healthAssessmentId: {
  type: mongoose.Schema.Types.ObjectId,
  ref: 'HealthAssessment'
},
privacyConsent: {
  dataCollection: {
    type: Boolean,
    default: false
  },
  dataSharing: {
    type: Boolean,
    default: false
  },
  marketing: {
    type: Boolean,
    default: false
  },
  consentTimestamp: Date
}
```

```
}, {
```

```
  timestamps: true
```

```
});
```

```
// Indexes
```

```
kioskSessionSchema.index({ sessionId: 1 });
```

```
kioskSessionSchema.index({ kioskId: 1, createdAt: -1 });
```

```
kioskSessionSchema.index({ userId: 1 });
```

```
kioskSessionSchema.index({ status: 1 });
```

```
kioskSessionSchema.index({ startTime: -1 });
```

```
// Virtual for session duration
```

```
kioskSessionSchema.virtual('sessionDuration').get(function() {
```

```
  if (this.endTime && this.startTime) {
```

```
    return this.endTime - this.startTime;
```

```
  }
```

```
  return Date.now() - this.startTime;
```

```
});
```

```
// Method to complete a step
```

```
kioskSessionSchema.methods.completeStep = function(step, data) {
```

```
  this.stepsCompleted.push({
```

```
    step,
```

```
    completedAt: new Date(),
```

```
    data
```

```
  });
```

```
  return this.save();
```

```
};
```

// Method to update current step

```
kioskSessionSchema.methods.updateStep = function(step) {  
  this.currentStep = step;  
  return this.save();  
};
```

// Method to add error

```
kioskSessionSchema.methods.addError = function(step, error) {  
  this.errors.push({  
    step,  
    error,  
    timestamp: new Date()  
  });  
  return this.save();  
};
```

// Method to end session

```
kioskSessionSchema.methods.endSession = function() {  
  this.endTime = new Date();  
  this.duration = this.endTime - this.startTime;  
  this.status = 'completed';  
  return this.save();  
};
```

// Static method to get active sessions

```
kioskSessionSchema.statics.getActiveSessions = function(kioskId) {  
  return this.find({
```

```
kioskId,  
status: 'active',  
startTime: { $gte: new Date(Date.now() - 30 * 60 * 1000) } // Last 30 minutes  
});  
};
```

// Static method to cleanup old sessions

```
kioskSessionSchema.statics.cleanupOldSessions = function() {  
  const cutoffDate = new Date(Date.now() - 24 * 60 * 60 * 1000); // 24 hours ago  
  return this.updateMany(  
    {  
      status: 'active',  
      startTime: { $lt: cutoffDate }  
    },  
    {  
      $set: {  
        status: 'abandoned',  
        endTime: new Date()  
      }  
    }  
  );  
};
```

```
module.exports = mongoose.model('KioskSession', kioskSessionSchema);
```

```
const jwt = require('jsonwebtoken');  
const User = require('../models/User');
```

```
const auth = async (req, res, next) => {  
  try {  
    const token = req.header('Authorization')?.replace('Bearer ', '');  
  
    if (!token) {  
      return res.status(401).json({  
        success: false,  
        message: 'Access denied. No token provided.'  
      });  
    }  
  
    const decoded = jwt.verify(token, process.env.JWT_SECRET);  
    const user = await User.findById(decoded.userId).select('-password');  
  
    if (!user || !user.isActive) {  
      return res.status(401).json({  
        success: false,  
        message: 'Invalid token or user not found.'  
      });  
    }  
  
    req.user = user;  
    next();  
  } catch (error) {  
    if (error.name === 'JsonWebTokenError') {  
      return res.status(401).json({  
        success: false,  
        message: 'Invalid token.'  
      });  
    }  
  }  
}
```

```
});  
}
```

```
if (error.name === 'TokenExpiredError') {  
  return res.status(401).json({  
    success: false,  
    message: 'Token expired.'  
  });  
}
```

```
res.status(500).json({  
  success: false,  
  message: 'Server error during authentication.'  
});  
}  
};
```

```
const optionalAuth = async (req, res, next) => {  
  try {  
    const token = req.header('Authorization')?.replace('Bearer ', '');  
  
    if (token) {  
      const decoded = jwt.verify(token, process.env.JWT_SECRET);  
      const user = await User.findById(decoded.userId).select('-password');  
  
      if (user && user.isActive) {  
        req.user = user;  
      }  
    }  
  }  
}
```

```
}

    next();
  } catch (error) {
    // For optional auth, we don't fail on token errors
    next();
  }
};
```

```
const requireRole = (roles) => {
  return (req, res, next) => {
    if (!req.user) {
      return res.status(401).json({
        success: false,
        message: 'Authentication required.'
      });
    }

    if (!roles.includes(req.user.role)) {
      return res.status(403).json({
        success: false,
        message: 'Insufficient permissions.'
      });
    }

    next();
  };
};
```

```
const generateToken = (userId) => {  
  return jwt.sign(  
    { userId },  
    process.env.JWT_SECRET,  
    { expiresIn: '24h' }  
  );  
};
```

```
module.exports = {  
  auth,  
  optionalAuth,  
  requireRole,  
  generateToken  
};
```

```
const errorHandler = (err, req, res, next) => {  
  let error = { ...err };  
  error.message = err.message;  
  
  // Log error  
  console.error('Error:', err);  
  
  // Mongoose bad ObjectId  
  if (err.name === 'CastError') {  
    const message = 'Resource not found';  
    error = { message, statusCode: 404 };  
  }  
}
```

// Mongoose duplicate key

```
if (err.code === 11000) {  
  const message = 'Duplicate field value entered';  
  error = { message, statusCode: 400 };  
}
```

// Mongoose validation error

```
if (err.name === 'ValidationError') {  
  const message = Object.values(err.errors).map(val => val.message).join(', ');  
  error = { message, statusCode: 400 };  
}
```

// JWT errors

```
if (err.name === 'JsonWebTokenError') {  
  const message = 'Invalid token';  
  error = { message, statusCode: 401 };  
}
```

```
if (err.name === 'TokenExpiredError') {  
  const message = 'Token expired';  
  error = { message, statusCode: 401 };  
}
```

// File upload errors

```
if (err.code === 'LIMIT_FILE_SIZE') {  
  const message = 'File too large';  
  error = { message, statusCode: 400 };  
}
```

```
}
```

```
if (err.code === 'LIMIT_UNEXPECTED_FILE') {
```

```
  const message = 'Unexpected file field';
```

```
  error = { message, statusCode: 400 };
```

```
}
```

```
res.status(error.statusCode || 500).json({
```

```
  success: false,
```

```
  message: error.message || 'Server Error',
```

```
  ...(process.env.NODE_ENV === 'development' && { stack: err.stack })
```

```
});
```

```
};
```

```
module.exports = { errorHandler };
```

```
const { body, param, query, validationResult } = require('express-validator');
```

```
const handleValidationErrors = (req, res, next) => {
```

```
  const errors = validationResult(req);
```

```
  if (!errors.isEmpty()) {
```

```
    return res.status(400).json({
```

```
      success: false,
```

```
      message: 'Validation failed',
```

```
      errors: errors.array()
```

```
    });
```

```
  }
```

```
  next();
```

```
};
```

```
// User registration validation
```

```
const validateUserRegistration = [  
  body('email')  
    .isEmail()  
    .normalizeEmail()  
    .withMessage('Please provide a valid email address'),  
  body('password')  
    .isLength({ min: 6 })  
    .withMessage('Password must be at least 6 characters long'),  
  body('firstName')  
    .trim()  
    .isLength({ min: 1 })  
    .withMessage('First name is required'),  
  body('lastName')  
    .trim()  
    .isLength({ min: 1 })  
    .withMessage('Last name is required'),  
  body('dateOfBirth')  
    .isISO8601()  
    .withMessage('Please provide a valid date of birth'),  
  body('gender')  
    .isIn(['male', 'female', 'other', 'prefer-not-to-say'])  
    .withMessage('Please select a valid gender'),  
  handleValidationErrors  
];
```

// User login validation

```
const validateUserLogin = [  
  body('email')  
    .isEmail()  
    .normalizeEmail()  
    .withMessage('Please provide a valid email address'),  
  body('password')  
    .notEmpty()  
    .withMessage('Password is required'),  
  handleValidationErrors  
];
```

// Health assessment validation

```
const validateHealthAssessment = [  
  body('assessmentType')  
    .optional()  
    .isIn(['general', 'cardiovascular', 'respiratory', 'diabetes', 'mental-health',  
    'comprehensive'])  
    .withMessage('Invalid assessment type'),  
  body('vitalSigns.bloodPressure.systolic')  
    .optional()  
    .isInt({ min: 50, max: 300 })  
    .withMessage('Systolic blood pressure must be between 50 and 300'),  
  body('vitalSigns.bloodPressure.diastolic')  
    .optional()  
    .isInt({ min: 30, max: 200 })  
    .withMessage('Diastolic blood pressure must be between 30 and 200'),  
  body('vitalSigns.heartRate')
```

```
.optional()  
.isInt({ min: 30, max: 220 })  
.withMessage('Heart rate must be between 30 and 220'),  
body('vitalSigns.temperature')  
.optional()  
.isFloat({ min: 95, max: 110 })  
.withMessage('Temperature must be between 95 and 110'),  
body('vitalSigns.oxygenSaturation')  
.optional()  
.isInt({ min: 70, max: 100 })  
.withMessage('Oxygen saturation must be between 70 and 100'),  
body('vitalSigns.weight')  
.optional()  
.isFloat({ min: 20, max: 500 })  
.withMessage('Weight must be between 20 and 500'),  
body('vitalSigns.height')  
.optional()  
.isFloat({ min: 100, max: 250 })  
.withMessage('Height must be between 100 and 250'),  
handleValidationErrors  
];
```

// Session validation

```
const validateSession = [  
  body('kioskId')  
    .notEmpty()  
    .withMessage('Kiosk ID is required'),  
  body('deviceInfo')
```

```
.optional()  
.isObject()  
.withMessage('Device info must be an object'),  
handleValidationErrors  
];
```

// ObjectId validation

```
const validateObjectId = (paramName) => [  
  param(paramName)  
    .isMongoid()  
    .withMessage(`Invalid ${paramName} ID`),  
  handleValidationErrors  
];
```

// Pagination validation

```
const validatePagination = [  
  query('page')  
    .optional()  
    .isInt({ min: 1 })  
    .withMessage('Page must be a positive integer'),  
  query('limit')  
    .optional()  
    .isInt({ min: 1, max: 100 })  
    .withMessage('Limit must be between 1 and 100'),  
  handleValidationErrors  
];
```

```
module.exports = {
```

```
validateUserRegistration,  
validateUserLogin,  
validateHealthAssessment,  
validateSession,  
validateObjectId,  
validatePagination,  
handleValidationErrors  
};
```

```
const express = require('express');  
const bcrypt = require('bcryptjs');  
const User = require('../models/User');  
const { generateToken, auth } = require('../middleware/auth');  
const { validateUserRegistration, validateUserLogin } =  
require('../middleware/validation');
```

```
const router = express.Router();
```

```
// @route POST /api/auth/register
```

```
// @desc Register a new user
```

```
// @access Public
```

```
router.post('/register', validateUserRegistration, async (req, res) => {
```

```
  try {
```

```
    const { email, password, firstName, lastName, dateOfBirth, gender, phoneNumber } =  
req.body;
```

```
// Check if user already exists
```

```
const existingUser = await User.findOne({ email });
```

```
if (existingUser) {
```

```
return res.status(400).json({
  success: false,
  message: 'User already exists with this email address'
});
}
```

// Create new user

```
const user = new User({
  email,
  password,
  firstName,
  lastName,
  dateOfBirth,
  gender,
  phoneNumber
});
```

```
await user.save();
```

// Generate token

```
const token = generateToken(user._id);
```

```
res.status(201).json({
  success: true,
  message: 'User registered successfully',
  data: {
    user: user.toJSON(),
    token
```

```

    }
  });
} catch (error) {
  console.error('Registration error:', error);
  res.status(500).json({
    success: false,
    message: 'Server error during registration'
  });
}
});

```

```

// @route POST /api/auth/login

```

```

// @desc Login user

```

```

// @access Public

```

```

router.post('/login', validateUserLogin, async (req, res) => {
  try {
    const { email, password } = req.body;

```

```

    // Find user by email

```

```

    const user = await User.findOne({ email });

```

```

    if (!user) {

```

```

      return res.status(401).json({

```

```

        success: false,

```

```

        message: 'Invalid email or password'

```

```

      });

```

```

    }

```

```

    // Check if user is active

```

```
if (!user.isActive) {  
  return res.status(401).json({  
    success: false,  
    message: 'Account is deactivated'  
  });  
}
```

// Check password

```
const isPasswordValid = await user.comparePassword(password);  
if (!isPasswordValid) {  
  return res.status(401).json({  
    success: false,  
    message: 'Invalid email or password'  
  });  
}
```

// Update last login

```
user.lastLogin = new Date();  
await user.save();
```

// Generate token

```
const token = generateToken(user._id);
```

```
res.json({  
  success: true,  
  message: 'Login successful',  
  data: {  
    user: user.toJSON(),
```

```
        token
    }
});
} catch (error) {
    console.error('Login error:', error);
    res.status(500).json({
        success: false,
        message: 'Server error during login'
    });
}
});
```

```
// @route GET /api/auth/me
// @desc Get current user
// @access Private
router.get('/me', auth, async (req, res) => {
    try {
        res.json({
            success: true,
            data: {
                user: req.user
            }
        });
    } catch (error) {
        console.error('Get user error:', error);
        res.status(500).json({
            success: false,
            message: 'Server error'
        });
    }
});
```

```

});

}

});

// @route PUT /api/auth/profile
// @desc Update user profile
// @access Private

router.put('/profile', auth, async (req, res) => {
  try {
    const allowedUpdates = [
      'firstName', 'lastName', 'phoneNumber', 'emergencyContact',
      'medicalHistory', 'medications', 'allergies', 'preferences'
    ];

    const updates = {};
    Object.keys(req.body).forEach(key => {
      if (allowedUpdates.includes(key)) {
        updates[key] = req.body[key];
      }
    });

    const user = await User.findByIdAndUpdate(
      req.user._id,
      updates,
      { new: true, runValidators: true }
    );

    res.json({

```

```

    success: true,
    message: 'Profile updated successfully',
    data: {
      user: user.toJSON()
    }
  });
} catch (error) {
  console.error('Profile update error:', error);
  res.status(500).json({
    success: false,
    message: 'Server error during profile update'
  });
}
});

// @route POST /api/auth/change-password
// @desc Change user password
// @access Private
router.post('/change-password', auth, async (req, res) => {
  try {
    const { currentPassword, newPassword } = req.body;

    if (!currentPassword || !newPassword) {
      return res.status(400).json({
        success: false,
        message: 'Current password and new password are required'
      });
    }
  }
});

```

```
if (newPassword.length < 6) {  
  return res.status(400).json({  
    success: false,  
    message: 'New password must be at least 6 characters long'  
  });  
}
```

```
const user = await User.findById(req.user._id);
```

```
// Verify current password
```

```
const isCurrentPasswordValid = await user.comparePassword(currentPassword);  
if (!isCurrentPasswordValid) {  
  return res.status(400).json({  
    success: false,  
    message: 'Current password is incorrect'  
  });  
}
```

```
// Update password
```

```
user.password = newPassword;  
await user.save();
```

```
res.json({  
  success: true,  
  message: 'Password changed successfully'  
});  
} catch (error) {
```

```

    console.error('Change password error:', error);
    res.status(500).json({
      success: false,
      message: 'Server error during password change'
    });
  }
});

module.exports = router;

const express = require('express');
const HealthAssessment = require('../models/HealthAssessment');
const KioskSession = require('../models/KioskSession');
const { auth, optionalAuth } = require('../middleware/auth');
const { validateHealthAssessment, validateObjectId, validatePagination } =
  require('../middleware/validation');

const router = express.Router();

// @route POST /api/health/assessment
// @desc Create a new health assessment
// @access Private
router.post('/assessment', auth, validateHealthAssessment, async (req, res) => {
  try {
    const assessmentData = {
      ...req.body,
      userId: req.user._id,
      kioskId: process.env.KIOSK_ID || 'kiosk-001'
    }
  }
});

```

```
};
```

```
// Generate unique session ID
```

```
const sessionId = `session_${Date.now()}_${Math.random().toString(36).substr(2, 9)}`;
```

```
assessmentData.sessionId = sessionId;
```

```
// Calculate BMI if weight and height are provided
```

```
if (assessmentData.vitalSigns?.weight && assessmentData.vitalSigns?.height) {
```

```
  const heightInMeters = assessmentData.vitalSigns.height / 100;
```

```
  assessmentData.vitalSigns.bmi = (
```

```
    assessmentData.vitalSigns.weight / (heightInMeters * heightInMeters)
```

```
  ).toFixed(1);
```

```
}
```

```
const assessment = new HealthAssessment(assessmentData);
```

```
await assessment.save();
```

```
res.status(201).json({
```

```
  success: true,
```

```
  message: 'Health assessment created successfully',
```

```
  data: {
```

```
    assessment,
```

```
    sessionId
```

```
  }
```

```
});
```

```
} catch (error) {
```

```
  console.error('Create assessment error:', error);
```

```
res.status(500).json({
  success: false,
  message: 'Server error during assessment creation'
});
}
});
```

```
// @route GET /api/health/assessments
```

```
// @desc Get user's health assessments
```

```
// @access Private
```

```
router.get('/assessments', auth, validatePagination, async (req, res) => {
```

```
  try {
```

```
    const page = parseInt(req.query.page) || 1;
```

```
    const limit = parseInt(req.query.limit) || 10;
```

```
    const skip = (page - 1) * limit;
```

```
    const assessments = await HealthAssessment.find({ userId: req.user._id })
```

```
      .sort({ createdAt: -1 })
```

```
      .skip(skip)
```

```
      .limit(limit)
```

```
      .populate('reviewedBy', 'firstName lastName');
```

```
    const total = await HealthAssessment.countDocuments({ userId: req.user._id });
```

```
    res.json({
```

```
      success: true,
```

```
      data: {
```

```
        assessments,
```

```

    pagination: {
      current: page,
      pages: Math.ceil(total / limit),
      total
    }
  }
});
} catch (error) {
  console.error('Get assessments error:', error);
  res.status(500).json({
    success: false,
    message: 'Server error'
  });
}
});

// @route GET /api/health/assessment/:id
// @desc Get specific health assessment
// @access Private
router.get('/assessment/:id', auth, validateObjectId('id'), async (req, res) => {
  try {
    const assessment = await HealthAssessment.findOne({
      _id: req.params.id,
      userId: req.user._id
    }).populate('reviewedBy', 'firstName lastName');

    if (!assessment) {
      return res.status(404).json({

```

```

        success: false,

        message: 'Health assessment not found'

    });
}

res.json({

    success: true,

    data: {

        assessment

    }

});

} catch (error) {

    console.error('Get assessment error:', error);

    res.status(500).json({

        success: false,

        message: 'Server error'

    });

}

});

// @route PUT /api/health/assessment/:id
// @desc Update health assessment
// @access Private

router.put('/assessment/:id', auth, validateObjectId('id'), validateHealthAssessment,
async (req, res) => {

    try {

        const assessment = await HealthAssessment.findOne({

            _id: req.params.id,

```

```
    userId: req.user._id
  });

  if (!assessment) {
    return res.status(404).json({
      success: false,
      message: 'Health assessment not found'
    });
  }

  // Update assessment data
  Object.keys(req.body).forEach(key => {
    if (req.body[key] !== undefined) {
      assessment[key] = req.body[key];
    }
  });

  // Recalculate BMI if weight or height changed
  if (req.body.vitalSigns?.weight || req.body.vitalSigns?.height) {
    if (assessment.vitalSigns?.weight && assessment.vitalSigns?.height) {
      const heightInMeters = assessment.vitalSigns.height / 100;
      assessment.vitalSigns.bmi = (
        assessment.vitalSigns.weight / (heightInMeters * heightInMeters)
      ).toFixed(1);
    }
  }

  await assessment.save();
```

```

    res.json({
      success: true,
      message: 'Health assessment updated successfully',
      data: {
        assessment
      }
    });
  } catch (error) {
    console.error('Update assessment error:', error);
    res.status(500).json({
      success: false,
      message: 'Server error during assessment update'
    });
  }
});

// @route POST /api/health/assessment/:id/complete
// @desc Mark assessment as completed
// @access Private
router.post('/assessment/:id/complete', auth, validateObjectId('id'), async (req, res) => {
  try {
    const assessment = await HealthAssessment.findOne({
      _id: req.params.id,
      userId: req.user._id
    });

    if (!assessment) {

```

```
return res.status(404).json({
  success: false,
  message: 'Health assessment not found'
});
}
```

```
assessment.status = 'completed';
assessment.completedAt = new Date();
await assessment.save();
```

```
res.json({
  success: true,
  message: 'Health assessment completed successfully',
  data: {
    assessment
  }
});
} catch (error) {
  console.error('Complete assessment error:', error);
  res.status(500).json({
    success: false,
    message: 'Server error'
  });
}
});
```

```
// @route GET /api/health/statistics
// @desc Get user's health statistics
```

```
// @access Private

router.get('/statistics', auth, async (req, res) => {

  try {

    const userId = req.user._id;

    // Get basic statistics

    const totalAssessments = await HealthAssessment.countDocuments({ userId });

    const completedAssessments = await HealthAssessment.countDocuments({

      userId,

      status: 'completed'

    });

    // Get latest assessment

    const latestAssessment = await HealthAssessment.findOne({ userId })

      .sort({ createdAt: -1 });

    // Get risk level distribution

    const riskDistribution = await HealthAssessment.aggregate([

      { $match: { userId, status: 'completed' } },

      {

        $group: {

          _id: '$aiAnalysis.urgencyLevel',

          count: { $sum: 1 }

        }

      }

    ]);

    // Get assessments by month (last 6 months)
```

```
const sixMonthsAgo = new Date();
sixMonthsAgo.setMonth(sixMonthsAgo.getMonth() - 6);

const monthlyAssessments = await HealthAssessment.aggregate([
  {
    $match: {
      userId,
      createdAt: { $gte: sixMonthsAgo }
    }
  },
  {
    $group: {
      _id: {
        year: { $year: '$createdAt' },
        month: { $month: '$createdAt' }
      },
      count: { $sum: 1 }
    }
  },
  { $sort: { '_id.year': 1, '_id.month': 1 } }
]);

res.json({
  success: true,
  data: {
    totalAssessments,
    completedAssessments,
    latestAssessment,
```

```

        riskDistribution,
        monthlyAssessments
    }
});
} catch (error) {
    console.error('Get statistics error:', error);
    res.status(500).json({
        success: false,
        message: 'Server error'
    });
}
});

```

```

// @route GET /api/health/assessments/public
// @desc Get public health assessments (for kiosk display)
// @access Public
router.get('/assessments/public', optionalAuth, async (req, res) => {
    try {
        // Only return basic, non-sensitive information
        const assessments = await HealthAssessment.find({ status: 'completed' })
            .select('assessmentType createdAt aiAnalysis.riskScore aiAnalysis.urgencyLevel')
            .sort({ createdAt: -1 })
            .limit(10);

        res.json({
            success: true,
            data: {
                assessments
            }
        });
    } catch (error) {
        console.error('Get public health assessments error:', error);
        res.status(500).json({
            success: false,
            message: 'Server error'
        });
    }
});

```

```

    }
  });
} catch (error) {
  console.error('Get public assessments error:', error);
  res.status(500).json({
    success: false,
    message: 'Server error'
  });
}
});

module.exports = router;

const express = require('express');
const KioskSession = require('../models/KioskSession');
const HealthAssessment = require('../models/HealthAssessment');
const { optionalAuth } = require('../middleware/auth');
const { validateSession, validatePagination } = require('../middleware/validation');

const router = express.Router();

// @route POST /api/kiosk/session/start
// @desc Start a new kiosk session
// @access Public
router.post('/session/start', validateSession, async (req, res) => {
  try {
    const { kioskId, deviceInfo, location } = req.body;

```

```
// Generate unique session ID
```

```
const sessionId = `kiosk_${Date.now()}_${Math.random().toString(36).substr(2, 9)}`;
```

```
const sessionData = {
```

```
  sessionId,
```

```
  kioskId,
```

```
  deviceInfo: deviceInfo || {},
```

```
  location: location || {},
```

```
  userId: req.user?._id // Optional if user is logged in
```

```
};
```

```
const session = new KioskSession(sessionData);
```

```
await session.save();
```

```
res.status(201).json({
```

```
  success: true,
```

```
  message: 'Kiosk session started successfully',
```

```
  data: {
```

```
    sessionId,
```

```
    session
```

```
  }
```

```
});
```

```
} catch (error) {
```

```
  console.error('Start session error:', error);
```

```
res.status(500).json({
```

```
  success: false,
```

```
  message: 'Server error during session start'
```

```
});
```

```

    }
  });

  // @route PUT /api/kiosk/session/:sessionId/step
  // @desc Update session step
  // @access Public
  router.put('/session/:sessionId/step', async (req, res) => {
    try {
      const { sessionId } = req.params;
      const { step, data } = req.body;

      const session = await KioskSession.findOne({ sessionId });
      if (!session) {
        return res.status(404).json({
          success: false,
          message: 'Session not found'
        });
      }

      // Update current step
      session.currentStep = step;

      // Complete the previous step if data is provided
      if (data) {
        await session.completeStep(step, data);
      } else {
        await session.save();
      }
    }
  });

```

```

    res.json({
      success: true,
      message: 'Session step updated successfully',
      data: {
        session
      }
    });
  } catch (error) {
    console.error('Update session step error:', error);
    res.status(500).json({
      success: false,
      message: 'Server error'
    });
  }
});

// @route POST /api/kiosk/session/:sessionId/error
// @desc Log session error
// @access Public
router.post('/session/:sessionId/error', async (req, res) => {
  try {
    const { sessionId } = req.params;
    const { step, error } = req.body;

    const session = await KioskSession.findOne({ sessionId });
    if (!session) {
      return res.status(404).json({

```

```
    success: false,  
    message: 'Session not found'  
  });  
}
```

```
await session.addError(step, error);
```

```
res.json({  
  success: true,  
  message: 'Error logged successfully'  
});  
} catch (error) {  
  console.error('Log session error:', error);  
  res.status(500).json({  
    success: false,  
    message: 'Server error'  
  });  
}  
});
```

```
// @route POST /api/kiosk/session/:sessionId/end
```

```
// @desc End kiosk session
```

```
// @access Public
```

```
router.post('/session/:sessionId/end', async (req, res) => {  
  try {  
    const { sessionId } = req.params;  
    const { healthAssessmentId } = req.body;
```

```
const session = await KioskSession.findOne({ sessionId });  
if (!session) {  
  return res.status(404).json({  
    success: false,  
    message: 'Session not found'  
  });  
}
```

```
// Update session with health assessment ID if provided  
if (healthAssessmentId) {  
  session.healthAssessmentId = healthAssessmentId;  
}
```

```
await session.endSession();
```

```
res.json({  
  success: true,  
  message: 'Session ended successfully',  
  data: {  
    session  
  }  
});
```

```
} catch (error) {  
  console.error('End session error:', error);  
  res.status(500).json({  
    success: false,  
    message: 'Server error'  
  });  
}
```

```
}  
});
```

```
// @route GET /api/kiosk/session/:sessionId
```

```
// @desc Get session details
```

```
// @access Public
```

```
router.get('/session/:sessionId', async (req, res) => {
```

```
  try {
```

```
    const { sessionId } = req.params;
```

```
    const session = await KioskSession.findOne({ sessionId })
```

```
      .populate('healthAssessmentId')
```

```
      .populate('userId', 'firstName lastName email');
```

```
    if (!session) {
```

```
      return res.status(404).json({
```

```
        success: false,
```

```
        message: 'Session not found'
```

```
      });
```

```
    }
```

```
    res.json({
```

```
      success: true,
```

```
      data: {
```

```
        session
```

```
      }
```

```
    });
```

```
  } catch (error) {
```

```

    console.error('Get session error:', error);

    res.status(500).json({
      success: false,
      message: 'Server error'
    });
  }
});

// @route GET /api/kiosk/sessions
// @desc Get kiosk sessions
// @access Public
router.get('/sessions', validatePagination, async (req, res) => {
  try {
    const page = parseInt(req.query.page) || 1;
    const limit = parseInt(req.query.limit) || 20;
    const skip = (page - 1) * limit;
    const kioskId = req.query.kioskId || process.env.KIOSK_ID;

    const query = kioskId ? { kioskId } : {};

    const sessions = await KioskSession.find(query)
      .sort({ createdAt: -1 })
      .skip(skip)
      .limit(limit)
      .populate('userId', 'firstName lastName email')
      .populate('healthAssessmentId', 'assessmentType status aiAnalysis.riskScore');

    const total = await KioskSession.countDocuments(query);
  }
});

```

```

res.json({
  success: true,
  data: {
    sessions,
    pagination: {
      current: page,
      pages: Math.ceil(total / limit),
      total
    }
  }
});
} catch (error) {
  console.error('Get sessions error:', error);
  res.status(500).json({
    success: false,
    message: 'Server error'
  });
}
});

// @route GET /api/kiosk/active-sessions
// @desc Get active kiosk sessions
// @access Public

router.get('/active-sessions', async (req, res) => {
  try {
    const kioskId = req.query.kioskId || process.env.KIOSK_ID;

```

```

const activeSessions = await KioskSession.getActiveSessions(kioskId);

res.json({
  success: true,
  data: {
    activeSessions,
    count: activeSessions.length
  }
});
} catch (error) {
  console.error('Get active sessions error:', error);
  res.status(500).json({
    success: false,
    message: 'Server error'
  });
}
});

// @route GET /api/kiosk/statistics
// @desc Get kiosk statistics
// @access Public
router.get('/statistics', async (req, res) => {
  try {
    const kioskId = req.query.kioskId || process.env.KIOSK_ID;
    const query = kioskId ? { kioskId } : {};

    // Get session statistics
    const totalSessions = await KioskSession.countDocuments(query);

```

```
const completedSessions = await KioskSession.countDocuments({
  ...query,
  status: 'completed'
});
```

```
const activeSessions = await KioskSession.countDocuments({
  ...query,
  status: 'active'
});
```

// Get sessions by status

```
const sessionsByStatus = await KioskSession.aggregate([
  { $match: query },
  {
    $group: {
      _id: '$status',
      count: { $sum: 1 }
    }
  }
]);
```

// Get average session duration

```
const avgDuration = await KioskSession.aggregate([
  {
    $match: {
      ...query,
      status: 'completed',
      duration: { $exists: true }
    }
  }
]);
```

```
},  
{  
  $group: {  
    _id: null,  
    avgDuration: { $avg: '$duration' }  
  }  
}  
]);
```

```
// Get sessions by hour (last 24 hours)
```

```
const last24Hours = new Date(Date.now() - 24 * 60 * 60 * 1000);
```

```
const sessionsByHour = await KioskSession.aggregate([
```

```
{  
  $match: {  
    ...query,  
    startTime: { $gte: last24Hours }  
  }  
},  
{  
  $group: {  
    _id: { $hour: '$startTime' },  
    count: { $sum: 1 }  
  }  
},  
{ $sort: { '_id': 1 } }  
]);
```

```
res.json({
```

```

    success: true,
    data: {
      totalSessions,
      completedSessions,
      activeSessions,
      completionRate: totalSessions > 0 ? (completedSessions / totalSessions *
100).toFixed(2) : 0,
      sessionsByStatus,
      averageDuration: avgDuration[0]?.avgDuration || 0,
      sessionsByHour
    }
  });
} catch (error) {
  console.error('Get kiosk statistics error:', error);
  res.status(500).json({
    success: false,
    message: 'Server error'
  });
}
});

```

```

// @route POST /api/kiosk/cleanup

```

```

// @desc Cleanup old sessions

```

```

// @access Public

```

```

router.post('/cleanup', async (req, res) => {

```

```

  try {

```

```

    const result = await KioskSession.cleanupOldSessions();

```

```
res.json({
  success: true,
  message: 'Old sessions cleaned up successfully',
  data: {
    modifiedCount: result.modifiedCount
  }
});
} catch (error) {
  console.error('Cleanup sessions error:', error);
  res.status(500).json({
    success: false,
    message: 'Server error during cleanup'
  });
}
});

module.exports = router;

const express = require('express');
const axios = require('axios');
const HealthAssessment = require('../models/HealthAssessment');
const { auth, optionalAuth } = require('../middleware/auth');

const router = express.Router();

// AI Analysis Service
class HealthAIAnalyzer {
  constructor() {
```

```
this.apiKey = process.env.OPENAI_API_KEY;
this.baseURL = 'https://api.openai.com/v1';
}
```

```
async analyzeHealthData(assessmentData) {
  try {
    // Prepare the health data for AI analysis
    const healthContext = this.prepareHealthContext(assessmentData);

    // Generate AI analysis using OpenAI
    const analysis = await this.generateHealthAnalysis(healthContext);

    // Process and structure the analysis
    return this.processAnalysis(analysis, assessmentData);
  } catch (error) {
    console.error('AI Analysis Error:', error);
    return this.getFallbackAnalysis(assessmentData);
  }
}
```

```
prepareHealthContext(assessmentData) {
  const { vitalSigns, symptoms, lifestyle, questionnaire } = assessmentData;

  let context = "Health Assessment Data:\n\n";

  // Vital Signs
  if (vitalSigns) {
    context += "Vital Signs:\n";
  }
}
```

```

    if (vitalSigns.bloodPressure) {
        context += ` - Blood Pressure:
${vitalSigns.bloodPressure.systolic}/${vitalSigns.bloodPressure.diastolic} mmHg\n`;
    }

    if (vitalSigns.heartRate) {
        context += ` - Heart Rate: ${vitalSigns.heartRate} bpm\n`;
    }

    if (vitalSigns.temperature) {
        context += ` - Temperature: ${vitalSigns.temperature}°F\n`;
    }

    if (vitalSigns.oxygenSaturation) {
        context += ` - Oxygen Saturation: ${vitalSigns.oxygenSaturation}%\n`;
    }

    if (vitalSigns.weight && vitalSigns.height) {
        context += ` - Weight: ${vitalSigns.weight} kg, Height: ${vitalSigns.height} cm\n`;
        if (vitalSigns.bmi) {
            context += ` - BMI: ${vitalSigns.bmi}\n`;
        }
    }

    context += "\n";
}

```

// Symptoms

```

if (symptoms && symptoms.primarySymptoms) {
    context += "Primary Symptoms:\n";
    symptoms.primarySymptoms.forEach(symptom => {
        context += ` - ${symptom.symptom} (${symptom.severity},
${symptom.duration})\n`;
    });
}

```

```

    context += "\n";
}

// Lifestyle
if (lifestyle) {
    context += "Lifestyle Factors:\n";
    context += ` - Exercise: ${lifestyle.exerciseFrequency}\n`;
    context += ` - Diet: ${lifestyle.dietQuality}\n`;
    context += ` - Sleep: ${lifestyle.sleepHours} hours\n`;
    context += ` - Stress: ${lifestyle.stressLevel}\n`;
    context += ` - Smoking: ${lifestyle.smokingStatus}\n`;
    context += ` - Alcohol: ${lifestyle.alcoholConsumption}\n`;
    context += "\n";
}

return context;
}

```

```

async generateHealthAnalysis(healthContext) {
    if (!this.apiKey) {
        throw new Error('OpenAI API key not configured');
    }
}

```

```
const prompt = `
```

As a medical AI assistant, analyze the following health assessment data and provide:

1. Risk Score (0-100): Overall health risk assessment
2. Risk Factors: List of identified risk factors

3. Recommendations: Specific, actionable health recommendations with priority levels
4. Predicted Conditions: Potential health conditions with probability estimates
5. Follow-up Requirements: Whether medical follow-up is needed and timeframe
6. Urgency Level: low, medium, high, or critical

Health Data:

`${healthContext}`

Please provide a structured analysis focusing on preventive healthcare and early intervention. Be specific and actionable in your recommendations.

Format your response as JSON with the following structure:

```
{
  "riskScore": number,
  "riskFactors": ["factor1", "factor2"],
  "recommendations": [
    {
      "category": "string",
      "priority": "low|medium|high|urgent",
      "title": "string",
      "description": "string",
      "actionItems": ["action1", "action2"]
    }
  ],
  "predictedConditions": [
    {
      "condition": "string",
      "probability": number,
```

```
    "confidence": "low|medium|high"
  }
],
"followUpRequired": boolean,
"followUpTimeframe": "string",
"urgencyLevel": "low|medium|high|critical"
}
`;
```

```
try {
  const response = await axios.post(
    `${this.baseURL}/chat/completions`,
    {
      model: "gpt-4",
      messages: [
        {
          role: "system",
          content: "You are a medical AI assistant specializing in preventive healthcare analysis. Provide accurate, evidence-based health assessments and recommendations. Always emphasize the importance of consulting healthcare professionals for medical decisions."
        },
        {
          role: "user",
          content: prompt
        }
      ],
      max_tokens: 2000,
      temperature: 0.3
    }
  );
```

```

    },
    {
      headers: {
        'Authorization': `Bearer ${this.apiKey}`,
        'Content-Type': 'application/json'
      }
    }
  );

  const content = response.data.choices[0].message.content;
  return JSON.parse(content);
} catch (error) {
  console.error('OpenAI API Error:', error.response?.data || error.message);
  throw error;
}
}

processAnalysis(analysis, assessmentData) {
  // Validate and clean the analysis data

  const processedAnalysis = {
    riskScore: Math.max(0, Math.min(100, analysis.riskScore || 0)),
    riskFactors: Array.isArray(analysis.riskFactors) ? analysis.riskFactors : [],
    recommendations: Array.isArray(analysis.recommendations) ?
analysis.recommendations : [],
    predictedConditions: Array.isArray(analysis.predictedConditions) ?
analysis.predictedConditions : [],
    followUpRequired: Boolean(analysis.followUpRequired),
    followUpTimeframe: analysis.followUpTimeframe || 'Not specified',
    urgencyLevel: ['low', 'medium', 'high', 'critical'].includes(analysis.urgencyLevel)
  };
}

```

```

    ? analysis.urgencyLevel
    : 'low'
  };

  // Add additional context-based recommendations
  this.addContextualRecommendations(processedAnalysis, assessmentData);

  return processedAnalysis;
}

addContextualRecommendations(analysis, assessmentData) {
  const { vitalSigns, lifestyle } = assessmentData;

  // BMI-based recommendations
  if (vitalSigns?.bmi) {
    const bmi = parseFloat(vitalSigns.bmi);
    if (bmi < 18.5) {
      analysis.recommendations.push({
        category: 'Nutrition',
        priority: 'medium',
        title: 'Underweight - Nutritional Support',
        description: 'Your BMI indicates you may be underweight. Consider consulting a nutritionist.',
        actionItems: ['Consult with a nutritionist', 'Increase caloric intake with healthy foods', 'Monitor weight regularly']
      });
    } else if (bmi > 30) {
      analysis.recommendations.push({
        category: 'Weight Management',

```

```

    priority: 'high',

    title: 'Obesity - Weight Management',

    description: 'Your BMI indicates obesity. Weight management is important for
overall health.',

    actionItems: ['Consult with a healthcare provider', 'Develop a weight loss plan',
'Increase physical activity', 'Modify diet']

  });
}
}

// Blood pressure recommendations
if (vitalSigns?.bloodPressure) {
  const { systolic, diastolic } = vitalSigns.bloodPressure;
  if (systolic >= 140 || diastolic >= 90) {
    analysis.recommendations.push({
      category: 'Cardiovascular',
      priority: 'urgent',
      title: 'High Blood Pressure',
      description: 'Your blood pressure readings are elevated. Immediate medical
attention may be required.',
      actionItems: ['Consult a healthcare provider immediately', 'Monitor blood pressure
regularly', 'Reduce sodium intake', 'Increase physical activity']
    });
    analysis.urgencyLevel = 'high';
  }
}

// Lifestyle recommendations
if (lifestyle?.exerciseFrequency === 'none') {

```

```
analysis.recommendations.push({
  category: 'Physical Activity',
  priority: 'medium',
  title: 'Increase Physical Activity',
  description: 'Regular exercise is important for overall health and disease prevention.',
  actionItems: ['Start with 10-15 minutes of daily walking', 'Gradually increase activity level', 'Find activities you enjoy']
});
}
```

```
getFallbackAnalysis(assessmentData) {
  // Fallback analysis when AI service is unavailable
  const { vitalSigns } = assessmentData;
  let riskScore = 30; // Default moderate risk
  let urgencyLevel = 'low';
  const recommendations = [];

  // Basic risk assessment based on vital signs
  if (vitalSigns?.bloodPressure) {
    const { systolic, diastolic } = vitalSigns.bloodPressure;
    if (systolic >= 140 || diastolic >= 90) {
      riskScore = 80;
      urgencyLevel = 'high';
      recommendations.push({
        category: 'Cardiovascular',
        priority: 'urgent',
        title: 'High Blood Pressure Detected',
```

```
        description: 'Your blood pressure readings are elevated. Please consult a healthcare provider.',
```

```
        actionItems: ['Consult healthcare provider', 'Monitor blood pressure', 'Reduce sodium intake']
```

```
    });
```

```
  }
```

```
}
```

```
if (vitalSigns?.bmi) {
```

```
  const bmi = parseFloat(vitalSigns.bmi);
```

```
  if (bmi > 30) {
```

```
    riskScore = Math.max(riskScore, 60);
```

```
    recommendations.push({
```

```
      category: 'Weight Management',
```

```
      priority: 'high',
```

```
      title: 'Weight Management Recommended',
```

```
      description: 'Your BMI indicates obesity. Weight management is important for health.',
```

```
      actionItems: ['Consult healthcare provider', 'Develop weight loss plan', 'Increase physical activity']
```

```
    });
```

```
  }
```

```
}
```

```
return {
```

```
  riskScore,
```

```
  riskFactors: ['Basic assessment completed - AI analysis unavailable'],
```

```
  recommendations,
```

```
  predictedConditions: [],
```

```
    followUpRequired: urgencyLevel === 'high',  
    followUpTimeframe: urgencyLevel === 'high' ? 'Within 1 week' : 'Within 1 month',  
    urgencyLevel  
  };  
}  
}
```

```
const aiAnalyzer = new HealthAIAnalyzer();
```

```
// @route POST /api/ai/analyze
```

```
// @desc Analyze health assessment data using AI
```

```
// @access Private
```

```
router.post('/analyze', auth, async (req, res) => {
```

```
  try {
```

```
    const { assessmentId } = req.body;
```

```
    if (!assessmentId) {
```

```
      return res.status(400).json({
```

```
        success: false,
```

```
        message: 'Assessment ID is required'
```

```
      });
```

```
    }
```

```
// Get the health assessment
```

```
const assessment = await HealthAssessment.findOne({
```

```
  _id: assessmentId,
```

```
  userId: req.user._id
```

```
});
```

```
if (!assessment) {  
  return res.status(404).json({  
    success: false,  
    message: 'Health assessment not found'  
  });  
}  
  
// Perform AI analysis  
  
const aiAnalysis = await aiAnalyzer.analyzeHealthData(assessment);  
  
// Update the assessment with AI analysis  
  
assessment.aiAnalysis = aiAnalysis;  
assessment.status = 'completed';  
assessment.completedAt = new Date();  
await assessment.save();  
  
res.json({  
  success: true,  
  message: 'Health assessment analyzed successfully',  
  data: {  
    assessment,  
    aiAnalysis  
  }  
});  
} catch (error) {  
  console.error('AI Analysis error:', error);  
  res.status(500).json({
```

```
    success: false,  
    message: 'Error during AI analysis'  
  });  
}  
});
```

```
// @route POST /api/ai/analyze-direct
```

```
// @desc Analyze health data directly without saving
```

```
// @access Private
```

```
router.post('/analyze-direct', auth, async (req, res) => {
```

```
  try {
```

```
    const assessmentData = req.body;
```

```
// Perform AI analysis
```

```
const aiAnalysis = await aiAnalyzer.analyzeHealthData(assessmentData);
```

```
res.json({
```

```
  success: true,
```

```
  message: 'Health data analyzed successfully',
```

```
  data: {
```

```
    aiAnalysis
```

```
  }
```

```
});
```

```
} catch (error) {
```

```
  console.error('Direct AI Analysis error:', error);
```

```
  res.status(500).json({
```

```
    success: false,
```

```
    message: 'Error during AI analysis'
```

```
});  
  
}  
  
});
```

```
// @route GET /api/ai/health-tips
```

```
// @desc Get AI-generated health tips
```

```
// @access Public
```

```
router.get('/health-tips', async (req, res) => {
```

```
  try {
```

```
    const { category, limit = 5 } = req.query;
```

```
    const categories = {
```

```
      general: 'general health and wellness',
```

```
      cardiovascular: 'heart health and cardiovascular disease prevention',
```

```
      diabetes: 'diabetes prevention and management',
```

```
      mental: 'mental health and stress management',
```

```
      nutrition: 'nutrition and healthy eating',
```

```
      exercise: 'physical activity and fitness'
```

```
    };
```

```
    const categoryContext = categories[category] || categories.general;
```

```
    if (!aiAnalyzer.apiKey) {
```

```
      // Return static tips if AI is not available
```

```
      const staticTips = [
```

```
        {
```

```
          title: 'Stay Hydrated',
```

```
          description: 'Drink at least 8 glasses of water daily to maintain proper hydration.',
```

```
        category: 'general'
    },
    {
        title: 'Regular Exercise',
        description: 'Aim for at least 150 minutes of moderate exercise per week.',
        category: 'exercise'
    },
    {
        title: 'Balanced Diet',
        description: 'Include fruits, vegetables, whole grains, and lean proteins in your diet.',
        category: 'nutrition'
    },
    {
        title: 'Adequate Sleep',
        description: 'Get 7-9 hours of quality sleep each night for optimal health.',
        category: 'general'
    },
    {
        title: 'Stress Management',
        description: 'Practice relaxation techniques like meditation or deep breathing.',
        category: 'mental'
    }
];

return res.json({
    success: true,
    data: {
```

```

    tips: staticTips.slice(0, parseInt(limit))
  }
});
}

```

const prompt = `Generate \${limit} practical health tips for \${categoryContext}. Each tip should be actionable and evidence-based. Format as JSON array with title, description, and category fields.`;

```

try {
  const response = await axios.post(
    `${aiAnalyzer.baseUrl}/chat/completions`,
    {
      model: "gpt-3.5-turbo",
      messages: [
        {
          role: "system",
          content: "You are a health and wellness expert. Provide practical, evidence-based health tips that are easy to implement."
        },
        {
          role: "user",
          content: prompt
        }
      ],
      max_tokens: 1000,
      temperature: 0.7
    },
    {

```

```
headers: {  
  'Authorization': `Bearer ${aiAnalyzer.apiKey}`,  
  'Content-Type': 'application/json'  
}  
}  
);
```

```
const tips = JSON.parse(response.data.choices[0].message.content);
```

```
res.json({  
  success: true,  
  data: {  
    tips  
  }  
});  
} catch (aiError) {  
  console.error('AI Tips generation error:', aiError);  
  // Fallback to static tips  
  res.json({  
    success: true,  
    data: {  
      tips: [  
        {  
          title: 'Stay Hydrated',  
          description: 'Drink at least 8 glasses of water daily to maintain proper hydration.',  
          category: 'general'  
        }  
      ]  
    }  
  });  
}
```

```
    }  
  });  
}  
} catch (error) {  
  console.error('Health tips error:', error);  
  res.status(500).json({  
    success: false,  
    message: 'Error generating health tips'  
  });  
}  
});  
  
module.exports = router;
```